

**TRABAJO INTEGRADOR CONCEPTOS Y PARADIGMAS DE LENGUAJES DE
PROGRAMACIÓN 2025**

Integrantes

Romagnoli Marcos Ezequiel Legajo: 24922/7

Cardone Gianuca Legajo 21252/7

Bolaño Julián. Legajo: 21589/2

Albertuz Matías Legajo 18630/0

Número de grupo: 15

Lenguajes: Java y JavaScript.

Nombre de la cátedra: Conceptos y Paradigmas de Lenguajes de Programación

INTRODUCCIÓN GENERAL

Java y **JavaScript**, aunque comparten similitudes en su sintaxis básica, son lenguajes con enfoques muy distintos: Java es orientado a objetos con tipado estático y fuerte, mientras que JavaScript es multiparadigma con tipado dinámico y débil. Este trabajo compara ambos lenguajes en cuanto al manejo de parámetros, los sistemas de tipos y la gestión de excepciones, destacando sus diferencias conceptuales y cómo estas impactan en su robustez, flexibilidad y aplicabilidad en diversos contextos de desarrollo.

A. Parámetros

En este tema se tratará los tipos y modos de parámetros soportados por los distintos lenguajes ya mencionados y se mencionarán las características más importantes en cada caso.

JAVA

```
public class Main
{
    public static void main(String[] args) {
        int suma = sumar(1,5,4);
        imprimir("Hola", suma);
    }

    // Pasaje de parametro por varargs
    public static int sumar (int... numeros)
    {
        int suma = 0;
        for (int num : numeros) {suma += num;} // suma los valores
        return suma;
    }

    // Pasaje de parametro por Referencia y primitivo
    public static void imprimir (String mensaje, int numero)
    { System.out.println(mensaje + numero); }
}
```

- En Java, los parámetros se ligán por **posición**, lo que significa que el orden en que se pasan los argumentos en una llamada a método debe coincidir exactamente con el orden de los parámetros en su declaración.
- Java no permite la ligadura por nombre.
- Todos los parámetros deben ser tipados explícitamente ya que Java utiliza un sistema de **tipado estático y obligatorio**, lo que significa que el compilador valida que los tipos de los argumentos coincidan con los parámetros definidos en la declaración del método.
- Para declarar parámetros en una función o método, primero especificamos el tipo de dato del parámetro, seguido por el nombre de la variable que actuará como el parámetro. Si necesitamos múltiples parámetros, los separamos con comas.
- Este lenguaje no cuenta con una sintaxis para especificar valores predeterminados para los parámetros, es decir que es necesario que se le asignen un valor.

Java admite diferentes tipos de parámetros:

1. **Tipos primitivos:** como `int`, `double`, `boolean`. Se pasan por **valor**, por lo tanto, cualquier modificación en el parámetro dentro del método **no afecta** a la variable original.
2. **Tipos por referencia:** como objetos, arrays, etc. También se pasan por **valor**, pero en este caso se copia la **referencia al objeto**, lo que permite modificar su contenido dentro del método.
3. **Parámetros varargs:** se usan con la sintaxis `tipo... nombre`, y permiten pasar una cantidad indefinida de argumentos del mismo tipo, que se tratan como un array dentro del método.

Otra característica sobre los parámetros en Java es la posibilidad de declararlos con la palabra clave “**final**”. Esto vuelve al parámetro **immutable**, es decir que su valor no puede ser modificado una vez que fue asignado. La manera de hacer esto es colocando la palabra clave antes de la declaración del tipo del parámetro.

Ej: `final tipo_de_variable nombre_de_variable`

JavaScript

```
1
2 // Define la función constructora "Persona"
3 function Persona(nombre, edad) {
4     this.nombre = nombre;
5     this.edad = edad;
6 }
7
8 // Crea un objeto "persona" utilizando la función constructora "Persona"
9 let persona = new Persona("Juan", 30);
10
11 // Imprime el objeto "persona" en la consola
12 console.log(persona); // Imprime "{ nombre: 'Juan', edad: 30 }"
13
```

- En JavaScript, los parámetros se ligan por **posición**, lo que significa que el orden en que se pasan los argumentos en una llamada a método debe coincidir exactamente con el orden de los parámetros en su declaración.
- JavaScript al ser un lenguaje de **tipado dinámico y débil**, los parámetros no requieren ser tipados explícitamente, esto permite que una función pueda recibir argumentos de cualquier tipo.
- Para declarar parámetros en una función en JavaScript, se indica el **nombre del parámetro** dentro de los paréntesis que siguen al nombre de la función. Si la función necesita **más de un parámetro**, se deben **separar por comas**.

Otra característica sobre los parámetros en JavaScript es que a partir de la versión ES6, JavaScript permite declarar valores predeterminados para los parámetros. Esto significa que si un parámetro no es proporcionado en la llamada, el parámetro toma el valor definido por defecto.

B- Sistema de Tipos

En este tema se comparan los sistemas de tipos de Java y JavaScript, destacando sus diferencias en cómo manejan y verifican los tipos de datos. Java emplea un tipado estático y fuerte, mientras que JavaScript utiliza un tipado dinámico y débil.

Java:

Java tiene un sistema de tipado estático y fuerte, lo que significa que los tipos de datos se definen y verifican en tiempo de compilación. Esto permite detectar errores antes de ejecutar el programa, aumentando la seguridad y confiabilidad del código y otorgando mayor mantenibilidad y claridad. Además de no permitir operaciones entre tipos diferentes de datos sin conversión explícita. La **desventaja** de esto es que otorga menor flexibilidad

```
public class Main {
    public static void main(String[] args) {
        int numero = 5;
        String texto = "10";
        int resultado = numero + texto; // ✗ Error de compilación: tipos incompatibles
        System.out.println("Resultado: " + resultado);
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        String numeroComoTexto = "8";
        int numero = Integer.parseInt(numeroComoTexto); // ✔ Conversión explícita
        int resultado = numero + 2;
        System.out.println("Suma: " + resultado);
    }
}
```

JavaScript:

JavaScript usa un sistema de tipado dinámico y débil, donde las variables pueden cambiar de tipo en tiempo de ejecución. Esta flexibilidad permite escribir código rápido y adaptativo sin necesidad de declarar tipos explícitamente y otorga una sintaxis mas simple y con menos codigo. Al ser de tipado débil permite las operaciones entre tipos de datos distintos sin una conversión explícita.

Al trabajar con tipado dinámico, las ligaduras se hacen al momento de la ejecución.

Ventajas: Mayor flexibilidad, permitiendo escribir funciones y estructuras mas genericas.

```
function sumar(a, b) {  
  return a + b;  
}  
console.log(sumar(3, 4));    // 7 (suma numérica)  
console.log(sumar("3", 4));  // "34" (concatenación automática)
```

Desventajas: Como no hay verificación de tipos en tiempo de compilación, podemos encontrarnos con errores en tiempo de ejecución.

```
function dividir(a, b) {  
  return a / b;  
}  
  
console.log(dividir("10", 2)); // ✅ Resultado: 5 (coerción automática)  
console.log(dividir("diez", 2)); // ! Resultado: NaN (no lanza error, pero falla silenciosamente)  
console.log(dividir(true, false)); // ! Resultado: Infinity (true → 1, false → 0)
```

C - Excepciones

En este apartado se analiza cómo Java y JavaScript gestionan las excepciones. Una excepción es una situación anómala que se da en la ejecución de un programa y que se supone que ocurre con poca frecuencia. Se presentarán ejemplos usando las estructuras **try-catch-finally**.

Java:

- Modelo de manejo: Utiliza el modelo de manejo por terminación.
- Cuenta con excepciones ya definidas (ejemplo `ArrayIndexOutOfBoundsException`), además permite crear nuevas excepciones.

Jerarquía de excepciones:

Java tiene una jerarquía rígida y organizada. Todas las excepciones derivan de `Throwable`. Se dividen en:

- **Checked exceptions** (ej: `IOException`) – El compilador obliga a manejarlas con `try-catch` o `throws`.
- **Unchecked exceptions** (ej: `NullPointerException`) – No es obligatorio manejarlas, aunque es recomendable para evitar errores en tiempo de ejecución.

Palabras claves

- `try` – bloque de código que puede lanzar un error
- `catch` – bloque que maneja el error
- `throw` – lanza una excepción

- **throws** – un metodo puede lanzar una excepcion pero no la maneja internamente con un try-catch sino que la delega al bloque que lo invoco.
- **finally** – bloque que siempre se ejecuta, haya o no error

```
public static void main(String[] args) {
    try { // (1) Entra al try
        int resultado = dividir(10, 0); // (2) Llama al metodo dividir
        System.out.println("Resultado: " + resultado);
    } catch (ArithmeticException e) { // (4) Entra al catch
        System.out.println("Error: No se puede dividir por cero."); // (5) imprime el mensaje
        // throw Exception ("Se lanza otra excepcion");
        // Si se lanza otra excepcion dentro de este catch, el programa finalizaria.
    }
    finally{
        System.out.println("Esta línea se ejecuta siempre");
    }
    System.out.println("Se ejecuta al finalizar el try");
}

public static int dividir(int a, int b) {
    if (b == 0) {
        throw new ArithmeticException("División por cero no permitida"); // (3) Se lanza la excepcion
    }
    return a / b;
}
```

Nota: Teniendo en cuenta el codigo, si se lanzara una excepcion no controlada por el catch, el programa finaliza- Y si dentro de un catch o finally ocurriera otra excepcion, el programa tambien finalizaria.

JavaScript:

- Modelo de manejo: Utiliza un modelo de manejo de excepciones por terminación.
- Cuenta con excepciones ya definidas (ejemplo SyntaxError), además permite crear nuevas excepciones.

Jerarquía de excepciones:

JavaScript tiene un sistema de errores flexible. Todos los errores son objetos que heredan de **Error**.. Algunos ejemplos comunes son: **TypeError**, **ReferenceError**, **SyntaxError**, **RangeError**.

Palabras claves

- **try** – bloque de código que puede lanzar un error
- **catch** – bloque que maneja el error
- **throw** – lanza una excepción
- **finally** – bloque que siempre se ejecuta, haya o no error

```
function main() {
    try { // (1) Entra al try
        let resultado = dividir(10, 0); // (2) Llama a la función dividir
        console.log("Resultado: " + resultado);
    } catch (e) { // (4) Entra al catch
        console.log("Error: No se puede dividir por cero."); // (5) imprime el mensaje
        // throw new Error("Se lanza otra excepción");
        // Si se lanza otra excepción dentro de este catch, el programa finalizaria.
    }
    finally {
        console.log("Esta línea se ejecuta siempre"); // bloque finally
    }
    console.log("Se ejecuta al finalizar el try");
}

function dividir(a, b) {
    if (b === 0) {
        throw new Error("División por cero no permitida"); // (3) Se lanza la excepción
    }
    return a / b;
}
```

Nota: Teniendo en cuenta el código, si se lanzara una excepción no controlada por el `catch`, el programa finaliza- Y si dentro de un `catch` o `finally` ocurriera otra excepción, el programa también finalizaría.

D - Conclusión:

Este trabajo nos sirvió para reforzar conceptos sobre Java, un lenguaje con el que ya teníamos experiencia previa en otras materias. A su vez, pudimos investigar y profundizar en JavaScript, que hasta el momento no lo habíamos utilizado de igual manera.

Aprendimos que en Java se puede pasar una lista variable de parámetros a un método, y descubrimos que Java utiliza excepciones chequeadas y no chequeadas.

Con respecto a JavaScript, pudimos entender y aplicar cómo funciona un lenguaje con tipado dinámico, y además descubrimos que permite asignar valores por defecto a los parámetros de una función en caso de que no se proporcionen, lo cual resulta muy útil.

Gracias a esta comparación práctica entre ambos lenguajes, logramos entender no solo sus diferencias sintácticas, sino también sus enfoques conceptuales. Esta experiencia consolidó nuestras habilidades en programación y nos brindó herramientas para seleccionar el lenguaje más adecuado según el contexto del problema a resolver.

Bibliografías

<https://docs.oracle.com/javase/tutorial/java/javaOO/arguments.html>

https://oregoom.com/java/parametros/#Tipos_de_parametros_en_Java

<https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Functions>

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Functions/Default_parameters

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Control_flow_and_error_handling

<https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/try...catch>

Louden, Kenneth. C. (2002). *Lenguajes de programación* (2.^a ed., pp. 173–236)

Louden, Kenneth C. (2002). *Lenguajes de programación* (2.^a ed., pp. 257–271)